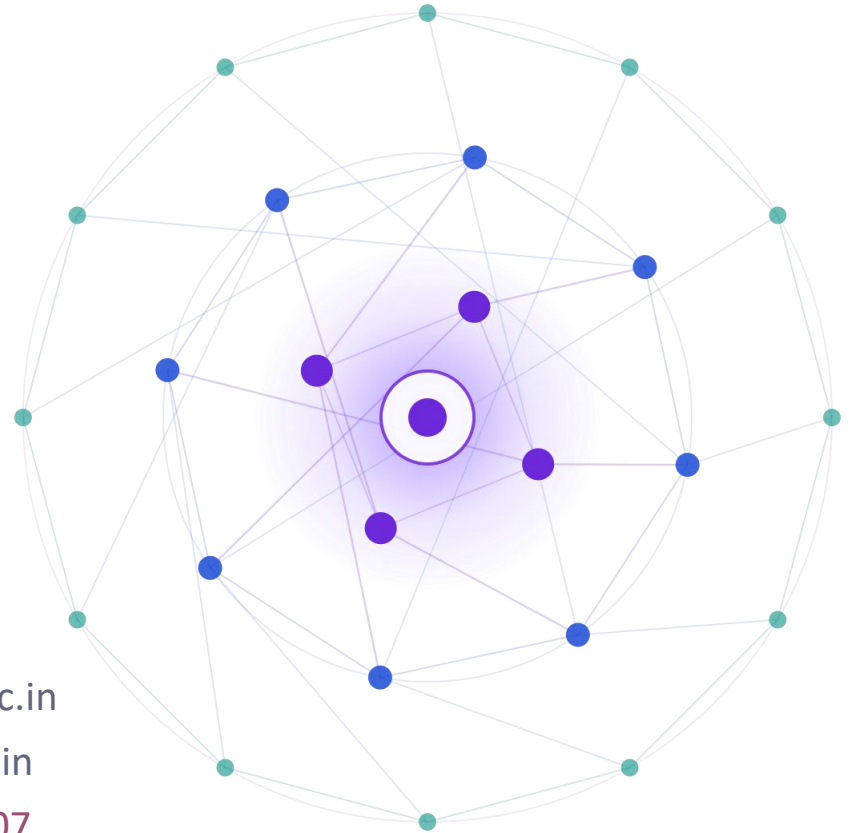


Generative AI Hackathon

3rd Edition 2026 – 27

- Project - debug907: natural-language code retrieval on a CPU
- Theme ID - 01, Agentic Code Intelligence
- Team Name – VITV_DEBUG907
- College Name – Vellore Institute of Technology, Vellore
- Member Name & Email 1- Kamneev Singh, kamneev.singh2025@vitstudent.ac.in
- Member Name & Email 2- Aadvik Chawla, aadvik.chawla2025@vitstudent.ac.in
- Submission Github link - https://github.com/kamneevsingh410/VIT_DEBUG907



Theme

Theme 01: Agentic Code Intelligence

The problem, in our words

A coding agent has to find the right code before it can fix or extend anything, and that step gets slower as the codebase grows. Theme 01 reduces it to one task: take a question in plain English and rank thousands of code snippets by how well they answer it.

Generating an answer is out of scope. We are building a ranker, and it has to run on a CPU, because retrieval is the fast step that sits in front of any LLM.

How it is scored

CoIR AppsRetrieval through MTEB: 3,765 problem statements, 8,765 Python solutions, exactly one correct solution per question. Metrics: NDCG@10 and MRR.

Goals, in priority order

P0 — Retrieval accuracy

Retrieve the relevant code snippets in response to a plain-English query.

P1 — Retrieval across versions

Rebuild indexes fast enough that a changing codebase is always searchable.

Bonus — Evolutionary retrieval

Search all versions at once, where near-identical snippets are hard to rank.

Constraint

Must run on CPU with minimal GPU use.

Existing Solutions & Gaps

Keyword search (BM25)

Matches words in the question against code. Scores 0.0053 NDCG@10 alone. Fused with a dense ranker it lowered the score by 0.027 to 0.043.

Stock model via MTEB registry

gte-modernbert-base as downloaded. Scores 0.31908 — the registry pins an old revision that silently uses the wrong pooling.

Same model, loaded correctly

Pinned revision, CLS pooling, 8,192 tokens. Scores 0.57673, level with the model card. Hub snippets still crowd the top: 19.3% of wrong #1 answers.

Cross-attention reranker or LLM

A second pass over the top results. An LLM cannot hold thousands of long snippets. The reranker cost -0.012 NDCG@10.

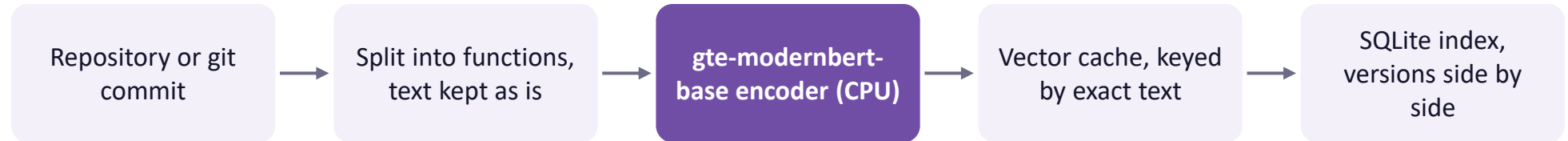
Re-embedding on every version

Rebuilds the whole index per commit — hopeless on a CPU. And the same function appears once per version: the top 10 held only 4 distinct functions without collapse.

Our Solutions & Architecture Diagram

Indexing

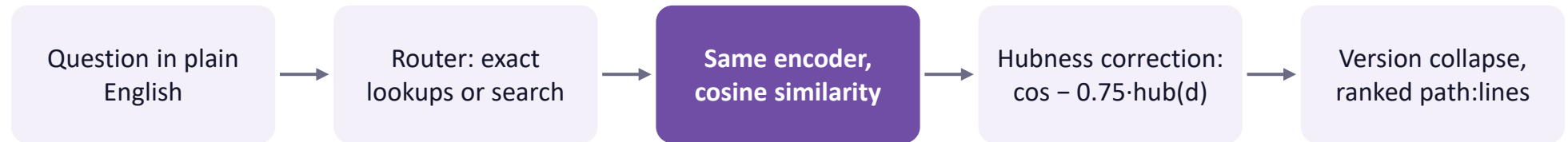
repo or git
commit



An unchanged function is never encoded again, in any version.

Asking

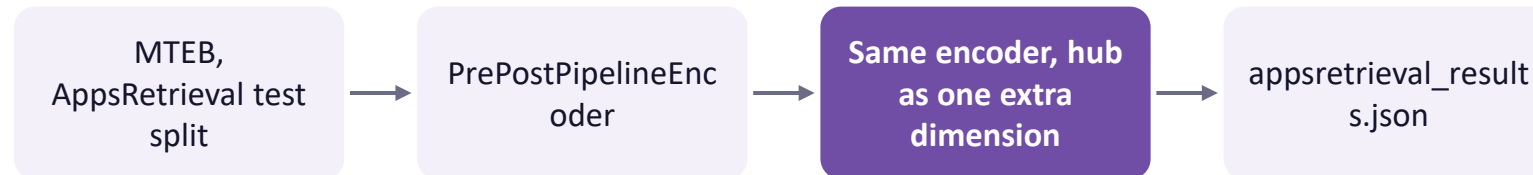
plain English



The router only takes short structural questions: 0 of 8,765 benchmark questions.

Screening

MTEB run



One encoder embeds both questions and code. No LLM in the retrieval path. Inside MTEB the correction is exact: +1 is appended to query vectors and $-0.75 \cdot \text{hub}(d)$ to document vectors, so MTEB's own dot product computes the corrected score.

How the score went from 0.11 to 0.61

jina-embeddings-v2-base-code (int8) → 0.10982

Our first artifact. The obvious code-specialist model.

gte-modernbert-base, 512 tokens → 0.48908

Model swap: the single biggest lever. On the 1,000-query sample gte scored 0.4906 vs jina 0.1434 and granite 0.1275.

Raise context to 1,024 tokens → 0.50431

Questions run 1,600 characters at the median; a quarter were cut off at 512 tokens. Worth +0.019 on the sample.

Add hubness correction → 0.53926

A small penalty for snippets that sit close to every question. +0.035 on the full split.

Embed raw document text → 0.60952

Embedding text exactly as given beat our cleaned "code view" by +0.070. CosQA, never touched, agreed: 37.62 → 43.06.

5.6× better than the first artifact

What did not work

Embed a summary instead of the code → -0.442

We tried generating a natural-language description of each snippet and embedding that. The encoder already understands code; a prose rewrite removes the very signal it looks for.

Strip input/output sections from queries → -0.260

Those sections are the most specific part of a competitive-programming problem statement. Removing them destroyed specificity.

BM25 + dense fusion → -0.027 to -0.043

BM25 alone scores 0.0053. Fused with the dense ranker it only added noise. The stronger the encoder, the less there is for keyword matching to add.

Cross-attention reranker → -0.012

A second-pass reranker over the top results. With a strong first-stage encoder there is little left to fix, and it hurt slightly.

The stronger the encoder, the more our hand-made processing got in its way.

Correcting for hub snippets

What we found

19.3% of wrong #1 answers came from a handful of "hub" snippets: generic solutions that sit close to every question.

The fix

Each document gets a hub score: its mean similarity to its 20 nearest training questions, excluding its own. The ranking score becomes

$$\cos(q, d) - 0.75 \cdot \text{hub}(d)$$

$k = 20$ and $\beta = 0.75$ were chosen on the 1,000-query sample.

Why it is legitimate

Training questions only, never a test question or a test label. Applied only when MTEB says the task is AppsRetrieval. Disclosed in the README, SHORTCOMINGS and REPORT.

Three checks on the final table

Gain on the full split

+0.0340

2,765 held-out test queries, never used for any choice

+0.0331 [+0.0266, +0.0404]

Same check — all 5,000 training answers removed from corpus

+0.0237 [+0.0176, +0.0297]

Does the hub score separate training answers? (AUC, 0.5 = random)

0.491 — no meaningful separation

P1: retrieval across versions

npm/cli — seconds to build the index

Cold build (2,506 functions, first time)

1,271 s

New release — most texts changed

104 s

New release — fewest texts changed

28 s

git sync per commit, no indexed code changed

12.8 s median

Only changed code is re-embedded

Every vector is keyed by the exact text that was embedded. A function that did not change is never encoded again, in any version. All versions live side by side in one SQLite index.

On npm releases

The cold build processes 2,506 texts. Each following release re-encodes only 22 to 170 texts — the functions that actually changed.

On git commits

debug907 sync <repo> indexes any commit as a version named by its short hash. It only reads the repository: nothing is checked out and no hooks are installed. --watch follows new commits.

Bonus: evolutionary retrieval

The problem

Three npm releases loaded, queried together: the top 10 held only 4 distinct functions. The same Publish class showed up once per release and filled the list.

What we built — version collapse

One row per function, labelled with every version it appears in. "=" means identical to the neighbour, "!=" means the code changed. :history N shows each version with a diff.

npm Publish, real result

v10.8.0 ≠ v10.8.1 = v10.8.2 ≠ v10.8.3 = v10.9.0

One row, not five. Changed twice across five releases.

Numbers

3 npm releases (10.8.0 – 10.9.0): 4 → 10 distinct functions in the top 10 with collapse on.

5 releases, 13,292 snippets in one index: 2 → 10.

Synthetic-version benchmark: 8.77 → 10.00 distinct solutions in the top 10.

How

Vectors keyed by exact text, so unchanged code is shared across versions and never re-embedded. Only the changed functions are re-encoded per release.

Demo & Product Walkthrough

python cli.py opens on npm's source (npm/cli 10.9.0, 2,663 functions). The tool always prints how it read the question.

Question → how the tool reads it

"how does npm decide a package is outdated"

→ ranked semantic search | Outdated command, outdated.js, ~80 ms

"where is otplease used"

→ usages of otplease | 19 lines in 13 files, file:line for each

"where is the npm registry fetch module used"

→ usages of npm-registry-fetch | name was written in words, resolved to the package

"who calls getCredentialsByURI"

→ call sites | adduser, login, profile, publish, get-identity

"where is the reify finish helper defined"

→ definition of reifyFinish | lib/utils/reify-finish.js line 6

"which files call validateLockfile before reifyFinish"

→ call order | one function, npm ci: lockfile line 68, finish line 125

"High" confidence label is right 82% of the time, "low" 15%. "Who calls X" is lexical and the tool says so.

Demo video (max 5 min): <https://youtu.be/vh2xW-vu988>

Tools and tech stack used

Encoder

gte-modernbert-base at a pinned revision, CLS pooling, 1,024 tokens, on CPU. One model for both questions and code.

Retrieval

Cosine nearest-neighbour search with the hubness correction. No LLM anywhere in the retrieval path.

Index

SQLite. Vectors keyed by the exact text embedded, so all versions share unchanged vectors.

Benchmark harness

MTEB 2.21.5, sentence-transformers, ColR AppsRetrieval test split.

Tool

Python terminal app (cli.py): plain-English router, exact lookups, ranked search, batch rank and eval, git sync. JS and TS functions extracted with a careful regex.

Packaging

Python 3.11, 3.12 and 3.14. Docker image is offline and non-root, model and dataset baked in; it fails unless it reproduces 0.6206.

Testing

364 tests, also run inside the Docker build.

Impact & Use case

Where it helps

Coding agents

Finding the right file is the first step of every bug fix. That step gets slower as the repository grows.

New developers

A plain-English question returns snippets with file and line, and the tool shows how it understood the question.

Codebases that change every commit

Only changed code is re-embedded. Results across versions collapse into one row per function.

Generalisation — held-out tasks, nothing tuned

StackOverflowQA

91.21 (model card: 91.2)

CosQA

43.06 (model card: 43.47)

WikiSQL

94.70

HumanEval / MBPP

84.48 / 80.73

CodeSearchNet JavaScript

80.84

CodeSearchNet Py / Go / Ruby / Java / PHP

90.26 / 95.89 / 85.19 / 91.59 / 88.87

npm, 127 questions of our own

NDCG@10 0.5975

DS1000 and FreshStack are the weak spots — explained on the limitations slide.

Innovation highlights, results and limitations

What we built that is new

Hubness correction

19.3% of wrong answers came from snippets close to everything. We score each document against its 20 nearest training questions and subtract 0.75× that score at query time.

Implemented as one extra vector dimension, so it is exact inside MTEB.

Text-keyed version index

Unchanged functions are never re-encoded. Cold build: 1,271 s. Each new release: 28–104 s.

Version collapse

One result row per function across all versions, labelled by which versions it appears in.

Results

Screening — NDCG@10 0.60952 · MRR@10 0.56320 · R@100 0.92457

5.6× over the first artifact (0.10982). +0.033 over the same model loaded correctly.

Right answer at #1 for 36.6% of questions, in the top 10 for 65.9%.

Limitations — we say these out loud

The guidelines' own three-snippet example ranks in reverse (within 0.018). Four-line placeholder code with no indentation. Tool flags it low confidence.

Hub table fitted to AppsRetrieval training questions. Passes three leakage checks.

"Who calls X" is a lexical lookup, not a call graph. JS is our weakest language (80.84).

What's next

1. A reranker built for near-duplicate snippets

DS1000 shows the failure: the right snippet is in the top 100 for 97.2% of questions but not ranked first. The generic cross-attention reranker we tried cost -0.012 . This needs a model trained specifically for near-identical code.

2. A real call graph

"Who calls X" is a lexical lookup today. It finds usages by name. A parsed AST call graph would make caller and call-order answers structurally exact rather than text-matched.

3. Fine-tuning on code with renamed identifiers

Renaming identifiers costs every model we tested 55–72% of its score, meaning the task leans heavily on function names. Training on renamed code would reduce that reliance.

Brownie points (differentiation)

Reproducible four ways

reproduce must hit 0.6206 on the sample. reproduce --full matches the artifact end-to-end. Samsung's own evaluation snippet (PrePostPipelineEncoder() with no arguments) gives 0.60952 exactly. score_csv.py --check re-scores the CSV in plain Python with no MTEB.

Security review

Seven findings fixed, each with a test written first: terminal escape injection from indexed code, symlink escape, output paths overwriting the artifact, git option injection, root Docker user, assert-based SQL check, shell line endings. Secrets are never indexed.

Optional pre-built index

Indexing takes 45–60 min on a laptop. A judge can import a 35.7 MB file instead, checked against a pinned SHA-256. verify-index re-encodes 200 random snippets from scratch (min cosine 1.0000000).

Tested across Python versions

Installed from an empty virtualenv on 3.11, 3.12 and 3.14. Vectors are bit-identical across all three. Docker is the fallback: offline, non-root, model and dataset baked in, 364 tests run during the build.

Negative results, all measured and shipped

BM25, fusion, rerank, query stripping and NL summaries were built, scored with a paired bootstrap, and left one switch away in the repo. Every design choice on these slides can be defended with a number.

The CSV scores identically to the tool

The batch rank path and the interactive path use the same encoder and correction. MTEB, the tool and the CSV all produce 0.60952 from the same weights.

Checklist- Updated on Public GitHub

- Working prototype code — public or shared GitHub repo (Y/N)
- README with reproducible setup instructions (Y/N)
- Demo video, max 5 minutes (YouTube or Drive link)
- Presentation file (PPT or PDF) (Y/N)
- Release tag PRISM_GENAI_HACKATHON_Y2026 on the final commit (Y/N)
- appsretrieval_results.json attached to the release (Y/N)

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India